

```
In [1]: # Imports
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import time
```

Problem Intro

In this problem, we will implement key components of the Muon optimizer in PyTorch. Then, we will compare Muon against SGD and AdamW empirically to demonstrate the benefits of Muon.

We will train a sample CNN architecture on CIFAR10 image data (initialized below).

```
In [2]: # Define a sample CNN for CIFAR-10
class CIFAR10CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(128 * 4 * 4, 512)
        self.fc2 = nn.Linear(512, 10)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = x.view(-1, 128 * 4 * 4)
        x = self.dropout(F.relu(self.fc1(x)))
        x = self.fc2(x)
        return x

# Dataset & loader with augmentation
transform_train = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
])
```

```

transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
])

# Increase batch size for more realistic training
batch_size = 128

train_ds = datasets.CIFAR10(root="./data", train=True, download=True,
transform=transform_train)
train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,
num_workers=2)

test_ds = datasets.CIFAR10(root="./data", train=False, download=True,
transform=transform_test)
test_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False,
num_workers=2)

```

100%|██████████| 170M/170M [00:03<00:00, 49.0MB/s]

Part 1: Implementing Newton-Schulz

The core of Muon is using matrix orthogonalization on a typical gradient update matrix. After orthogonalization:

- * Singular values become more uniform
- * Updates act across all directions in parameter space
- * The neural network can utilize its full parameter capacity (because parameters are all receiving nontrivial gradients)

In Muon, matrix orthogonalization is done using Newton-Schulz. Newton-Schulz relies on iteration using an odd matrix polynomial. In this problem, we consider a very simple cubic polynomial, but many odd polynomials can be used (state-of-the-art implementations use specific, tuned quintic polynomial).

Complete the following code to implement Newton-Schulz, using the matrix polynomial in the comments.

```

In [3]: def newton_schulz_orthogonalize(X: torch.Tensor, num_iters: int):
        """
        Apply Newton-Schulz iterations to approximate orthogonalization.

        This function applies the polynomial  $f(X) = (3X - X^3)/2$  repeatedly to a
        normalized matrix,
        which gradually forces all singular values to 1 while preserving singular

```

vectors.

Args:

X (torch.Tensor): Input matrix to orthogonalize

num_iters (int): Number of Newton-Schulz iterations

Returns:

torch.Tensor: Orthogonalized matrix

"""

dtype = X.dtype

Use bfloat16 for potential speed/memory savings during the iterations

X = X.bfloat16()

Recall from prior homeworks that we can transpose the matrix to speed up computation.

transposed = False

if X.size(-2) < X.size(-1):

transposed = True

X = X.mT

Ensure spectral norm is at most sqrt(3)

norm = torch.linalg.norm(X, dim=(-2, -1), keepdim=True)

X = torch.div(X, norm + 1e-7) * (3**0.5)

#####

##

TODO: YOUR CODE HERE

Apply the Newton-Schulz iteration: $f(X) = (3X - X^3) / 2$

where X^3 means $X @ X^T @ X$ (matrix multiplication, not element-wise)

#####

##

for _ in range(num_iters):

Compute $X @ X^T @ X$ (this is X^3 in the matrix sense for orthogonalization)

For a matrix X, X^3 in NS iteration is $X @ X.T @ X$

X = (3 * X - X @ X.mT @ X) / 2

#####

##

if transposed:

X = X.mT

return X.to(dtype)

Question 1

Notice that in the above implementation, we scale the spectral norm to be at most $\sqrt{3}$. **Can**

you explain why we choose this particular scaling?

(Hint: Inspect the roots of the cubic polynomial. What is the connection between the roots and the convergence properties of the singular values? You can refer to Discussion 4 for the answer)

answer question 1 in the part below

Looking at the notebook, **Question 1** asks:

> Notice that in the above implementation, we scale the spectral norm to be at most $\sqrt{3}$. **Can you explain why we choose this particular scaling?**

Answer to Question 1

We scale the spectral norm to be at most $\sqrt{3}$ because of the **convergence properties of the cubic polynomial** $f(\sigma) = \frac{3\sigma - \sigma^3}{2}$.

Analysis of the polynomial's roots and fixed points:

1. **Fixed points:** Setting $f(\sigma) = \sigma$:

$$\frac{3\sigma - \sigma^3}{2} = \sigma$$

$$3\sigma - \sigma^3 = 2\sigma$$

$$\sigma - \sigma^3 = 0$$

$$\sigma(1 - \sigma^2) = 0$$

This gives fixed points at $\sigma = 0, \pm 1$.

2. **Roots of $f(\sigma) = 0$:**

$$\frac{3\sigma - \sigma^3}{2} = 0$$

$$\sigma(3 - \sigma^2) = 0$$

This gives roots at $\sigma = 0, \pm\sqrt{3}$.

3. **Convergence basin:** For the iteration to converge to $\sigma = 1$ (the desired orthogonalization target), the initial singular value must be in the **basin of attraction** of the fixed point at 1.

- For $|\sigma| < \sqrt{3}$, repeated application of f drives $\sigma \rightarrow 1$ (or -1)
- For $|\sigma| > \sqrt{3}$, the iteration **diverges** to infinity
- At exactly $|\sigma| = \sqrt{3}$, $f(\pm\sqrt{3}) = 0$, which then stays at 0

Conclusion: By scaling the matrix so that all singular values are at most $\sqrt{3}$, we ensure that every singular value lies within the convergence basin of the Newton-Schulz iteration. This guarantees that after sufficient iterations, all singular values will converge to 1, achieving orthogonalization.

Part 2: Implementing Muon Update

Now, we implement the update in a Muon optimizer. Given parameter matrix W with momentum matrix M , the pseudocode for the Muon update proceeds as follows:

```
,
d_out, d_in = M.shape
```

Apply Newton-Schulz orthogonalization

```
M ← newton_schulz_orthogonalize(M, ns_iters)
```

Apply muP scaling factor for consistent update magnitude

```
M ← M · sqrt(max(1, d_out / d_in))
,
```

Then, the Muon update is used later to update the parameters W :

```
,
```

Update the parameter matrix

```
W ← W - lr * M
,
```

Complete the following code to implement the Muon update following the above pseudocode.

```
In [4]: import torch.optim as optim

def muon_update(grad, momentum, beta=0.95, ns_iters=5):
    momentum.lerp_(grad, 1 - beta) # momentum = beta * momentum + (1-beta) *
    grad
    update = momentum.clone()

    # If the parameter is a convolutional kernel, then flatten to a 2D matrix
    original_shape = update.shape
    reshaped = False
    if update.ndim > 2:
        reshaped = True
        update = update.view(update.size(0), -1)

#####
##
# TODO: YOUR CODE HERE

#####
##
# Get dimensions: d_out x d_in
d_out, d_in = update.shape

# Apply Newton-Schulz orthogonalization
update = newton_schulz_orthogonalize(update, ns_iters)

# Apply muP scaling factor for consistent update magnitude
#  $M \leftarrow M \cdot \sqrt{\max(1, d_{\text{out}} / d_{\text{in}})}$ 
update = update * (max(1, d_out / d_in) ** 0.5)

#####
##

# Restore shape if needed
if reshaped:
    update = update.view(original_shape)

return update

class Muon(optim.Optimizer):
    def __init__(self, params, lr=0.01, beta=0.95, ns_iters=5,
                 weight_decay=0):

        defaults = dict(lr=lr, beta=beta, ns_iters=ns_iters,
                        weight_decay=weight_decay)
        super().__init__(params, defaults)
```

```

@torch.no_grad()
def step(self, closure=None):
    for group in self.param_groups:
        lr = group['lr']
        beta = group['beta']
        ns_iters = group['ns_iters']
        weight_decay = group['weight_decay']

        for p in group['params']:
            if p.grad is None:
                continue

            grad = p.grad
            # Get state for this parameter
            state = self.state[p]
            # Initialize momentum buffer if it doesn't exist
            if 'momentum' not in state:
                state['momentum'] = torch.zeros_like(grad)

            # Apply weight decay directly to parameters (AdamW style)
            if weight_decay != 0:
                p.mul_(1 - lr * weight_decay)

            # Apply newton_schulz if parameter is a matrix
            if p.ndim >= 2:
                update = muon_update(grad, state['momentum'],
                                     beta=beta, ns_iters=ns_iters)
                # Apply update to parameters
                p.add_(update, alpha=-lr)
            else:
                # For non-matrix parameters, i.e. bias, use standard
momentum update
                momentum = state['momentum']
                momentum.mul_(beta).add_(grad)
                p.add_(momentum, alpha=-lr)

    return None

```

Question 2

Note that Muon requires that parameters are 2D matrices of shape $d_{\text{out}} \times d_{\text{in}}$. However, we know that parameters that are convolutional kernels have shape $c_{\text{out}} \times c_{\text{in}} \times k \times k$ where c denotes number of channels and k is kernel size.

Modern implementations of convolutional layers will transform an input image $\mathbf{x}$ of shape $c_{\text{in}} \times h \times w$ to $\mathbf{x}'$ such that each column has size $c_{\text{in}} \times k$ and corresponds to one flattened "receptive field" of the image (or one patch of the image that a convolutional filter passes over to compute one pixel in the output).

Given this fact, **how do we modify the convolutional kernel into a $d_{\text{out}} \times d_{\text{in}}$ matrix C such that the output of the convolutional layer can be expressed as $C \mathbf{x}$ '.**

answer question 2

Looking at the notebook, **Question 2** asks:

- > Note that Muon requires that parameters are 2D matrices of shape $d_{\text{out}} \times d_{\text{in}}$. However, we know that parameters that are convolutional kernels have shape $c_{\text{out}} \times c_{\text{in}} \times k \times k$ where c denotes number of channels and k is kernel size.
- >
- > Modern implementations of convolutional layers will transform an input image $\mathbf{x}$ of shape $c_{\text{in}} \times h \times w$ to $\mathbf{x}'$ such that each column has size $c_{\text{in}} \cdot k \cdot k$ and corresponds to one flattened "receptive field" of the image (or one patch of the image that a convolutional filter passes over to compute one pixel in the output).
- >
- > Given this fact, **how do we modify the convolutional kernel into a $d_{\text{out}} \times d_{\text{in}}$ matrix C such that the output of the convolutional layer can be expressed as $C \mathbf{x}$ '.**

Answer to Question 2

To convert a convolutional kernel of shape $c_{\text{out}} \times c_{\text{in}} \times k \times k$ into a 2D matrix C of shape $d_{\text{out}} \times d_{\text{in}}$, we **reshape (flatten) the kernel** as follows:

$$C = \text{reshape}(\text{kernel}, (c_{\text{out}}, c_{\text{in}} \cdot k \cdot k))$$

This gives us:

- $d_{\text{out}} = c_{\text{out}}$ (the number of output channels / filters)
- $d_{\text{in}} = c_{\text{in}} \cdot k \cdot k$ (the flattened size of each filter)

Explanation:

Each convolutional filter has shape $c_{\text{in}} \times k \times k$, which represents the weights applied to one receptive field patch of the input. By flattening each filter into a row vector of length $c_{\text{in}} \cdot k \cdot k$, we get a matrix C where:

- Each **row** of C corresponds to one output channel's filter (flattened)
- Each **column** of $\mathbf{x}'$ corresponds to one flattened receptive field patch

The convolution operation then becomes a simple matrix multiplication:

$$\text{output} = C \mathbf{x}'$$

where $\mathbf{x}'$ has shape $(c_{in} \cdot k \cdot k) \times n_{patches}$ and $n_{patches}$ is the number of spatial positions in the output.

This is exactly what the code does with `update.view(update.size(0), -1)` — it keeps the first dimension (c_{out}) and flattens everything else ($c_{in} \times k \times k$) into the second dimension.

Part 3: Empirical Evaluation of Muon

Now, we'll train the CNN network on the CIFAR10 dataset using our Muon implementation, comparing performance on the test set against other popular optimizers in SGD and AdamW.

First, in addition to SGD and AdamW, we consider two additional baseline optimizers that will help us better interpret our results:

- * MuonSVD: Rather than using Newton-Schulz, we orthogonalize the momentum using SVD on the momentum matrix $M = U \Sigma V^T$ and computing UV^T .

- * AdamWMuP: We add the muP scaling on top of the AdamW optimizer. This is meant to help us better understand how much of the Muon performance is due to the orthogonalization step, and how much is simply from the muP scaling.

The cell below implements these two additional optimizers. You do not have to implement anything.

```
In [5]: import torch.optim as optim

def muon_update_svd(grad, momentum, beta=0.95):
    momentum.lerp_(grad, 1 - beta) # momentum = beta * momentum + (1-beta) * grad
    update = momentum.clone()

    # If the parameter is a convolutional kernel, then flatten to a 2D matrix
    original_shape = update.shape
    reshaped = False
    if update.ndim > 2:
```

```

        reshaped = True
        update = update.view(update.size(0), -1)

    # Orthogonalization via SVD – specify full_matrices=False for reduced SVD
    U, _, Vh = torch.linalg.svd(update, full_matrices=False)
    update = torch.matmul(U, Vh)

    # Apply muP scaling
    update.mul_(max(1, update.size(-2) / update.size(-1))**0.5)

    # Restore shape if needed
    if reshaped:
        update = update.view(original_shape)

    return update

class MuonSVD(optim.Optimizer):
    def __init__(self, params, lr=0.01, beta=0.95, weight_decay=0):

        defaults = dict(lr=lr, beta=beta, weight_decay=weight_decay)
        super().__init__(params, defaults)

    @torch.no_grad()
    def step(self, closure=None):
        for group in self.param_groups:
            lr = group['lr']
            beta = group['beta']
            weight_decay = group['weight_decay']

            for p in group['params']:
                if p.grad is None:
                    continue

                grad = p.grad
                # Get state for this parameter
                state = self.state[p]
                # Initialize momentum buffer if it doesn't exist
                if 'momentum' not in state:
                    state['momentum'] = torch.zeros_like(grad)

                # Apply weight decay directly to parameters (AdamW style)
                if weight_decay != 0:
                    p.mul_(1 - lr * weight_decay)

                # Apply newton_schulz if parameter is a matrix
                if p.ndim >= 2:
                    update = muon_update_svd(grad, state['momentum'],
beta=beta)

                    # Apply update to parameters
                    p.add_(update, alpha=-lr)
                else:
                    # For non-matrix parameters, i.e. bias, use standard

```

momentum update

```
momentum = state['momentum']
momentum.mul_(beta).add_(grad)
p.add_(momentum, alpha=-lr)
```

return None

```
class AdamWMuP(optim.Optimizer):
```

```
    def __init__(self, params, lr=0.01, betas=(0.9, 0.999), weight_decay=0):
        defaults = dict(lr=lr, betas=betas, weight_decay=weight_decay)
        super().__init__(params, defaults)
```

```
@torch.no_grad()
```

```
    def step(self):
```

```
        for group in self.param_groups:
            lr = group['lr']
            weight_decay = group['weight_decay']
```

```
        for p in group['params']:
            grad = p.grad.data
```

```
            state = self.state[p]
            if len(state) == 0: # Initialization
                state["step"] = torch.tensor(0.0)
                state['momentum'] = torch.zeros_like(p)
                state['variance'] = torch.zeros_like(p)
```

```
            if weight_decay != 0:
                p.mul_(1 - lr * weight_decay)
```

```
            state['step'] += 1
            m = state['momentum']
            m.lerp_(grad, 1 - group["betas"][0])
            v = state['variance']
            v.lerp_(grad**2, 1 - group["betas"][1])
```

```
            m_hat = m / (1 - group["betas"][0]**state['step'])
            v_hat = v / (1 - group["betas"][1]**state['step'])
            u = m_hat / (torch.sqrt(v_hat) + 1e-16)
```

```
            if p.ndim >= 2:
```

```
                # If the parameter is a convolutional kernel, then flatten
```

to a 2D matrix

```
                original_shape = u.shape
                reshaped = False
                if u.ndim > 2:
```

```
                    u = u.view(u.shape[0], -1) # keep first dim, flatten
```

the rest

```
                reshaped = True
```

```
            u.mul_(max(1, u.size(-2) / u.size(-1))**0.5)
```

```

        # Unflatten back to convolutional kernel
        if reshaped:
            u = u.view(original_shape)

        p.add_(u, alpha=-lr)

    return None

```

If you performed hyperparameter sweeping (optional part below), then replace the default hyperparameters with the values you found from your sweep. Then, run the following cell to investigate how good Muon is relative to other baseline optimizers. The cell should take less than 10 minutes if you use a GPU runtime on Colab.

```

In [6]: # --- Define the optimizers you want to compare ---
optimizers_dict = {
    "Muon": lambda params: Muon(params, lr=1e-2, weight_decay=0),
    "SGD": lambda params: torch.optim.SGD(params, lr=1e-2, momentum=0.9,
weight_decay=1e-4),
    "AdamW": lambda params: torch.optim.AdamW(params, lr=1e-3,
weight_decay=1e-3),
    "MuonSVD": lambda params: MuonSVD(params, lr=1e-2, weight_decay=0),
    "AdamWMuP": lambda params: AdamWMuP(params, lr=1e-3, weight_decay=1e-3)
}

device = "cuda" if torch.cuda.is_available() else "cpu"
criterion = nn.CrossEntropyLoss()

# Function to evaluate model
def evaluate(model, dataloader):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in dataloader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    return 100 * correct / total

results = {} # store loss curves
accuracy_results = {} # store accuracy curves

# --- Train for each optimizer ---
for opt_name, opt_fn in optimizers_dict.items():
    print(f"\n--- Training with {opt_name} ---")
    model = CIFAR10CNN().to(device) # re-init model each time
    optimizer = opt_fn(model.parameters())

```

```

# Learning rate scheduler
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=5)

losses = []
accuracies = []
epoch_times = []

for epoch in range(1, 6): # Train for 5 epochs
    model.train()
    epoch_start_time = time.time()
    total_loss = 0

    for i, (x, y) in enumerate(train_loader):
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()
        out = model(x)
        loss = criterion(out, y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    # Print progress
    if (i+1) % 100 == 0:
        print(f'Epoch [{epoch}/5], Step [{i+1}/{len(train_loader)}],
Loss: {loss.item():.4f}')

    epoch_end_time = time.time()
    epoch_duration = epoch_end_time - epoch_start_time
    epoch_times.append(epoch_duration)

# Evaluate
test_acc = evaluate(model, test_loader)

avg_loss = total_loss / len(train_loader)
losses.append(avg_loss)
accuracies.append(test_acc)

print(f"{opt_name} | Epoch {epoch}, avg loss: {avg_loss:.4f}, test
acc: {test_acc:.2f}%, time: {epoch_duration:.2f} seconds")

# Update learning rate
scheduler.step()

results[opt_name] = losses
accuracy_results[opt_name] = accuracies

# Calculate and print total training time
total_time = sum(epoch_times)
print(f"{opt_name} | Total training time: {total_time:.2f} seconds")

# --- Plot results ---
# 1. Loss vs Epoch

```

```

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
for opt_name, losses in results.items():
    plt.plot(range(1, len(losses)+1), losses, label=opt_name, marker="o")
plt.xlabel("Epoch")
plt.ylabel("Average Loss")
plt.title("Training Loss vs Epoch for Different Optimizers")
plt.legend()
plt.grid(True)

# 2. Accuracy vs Epoch
plt.subplot(1, 2, 2)
for opt_name, accuracies in accuracy_results.items():
    plt.plot(range(1, len(accuracies)+1), accuracies, label=opt_name,
marker="o")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.title("Test Accuracy vs Epoch for Different Optimizers")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

```

```

--- Training with Muon ---
Epoch [1/5], Step [100/391], Loss: 1.8333
Epoch [1/5], Step [100/391], Loss: 1.8333
Epoch [1/5], Step [200/391], Loss: 1.5170
Epoch [1/5], Step [200/391], Loss: 1.5170
Epoch [1/5], Step [300/391], Loss: 1.1746
Epoch [1/5], Step [300/391], Loss: 1.1746
Muon | Epoch 1, avg loss: 1.5274, test acc: 62.81%, time: 20.02 seconds
Muon | Epoch 1, avg loss: 1.5274, test acc: 62.81%, time: 20.02 seconds
Epoch [2/5], Step [100/391], Loss: 1.0727
Epoch [2/5], Step [100/391], Loss: 1.0727
Epoch [2/5], Step [200/391], Loss: 1.0392
Epoch [2/5], Step [200/391], Loss: 1.0392
Epoch [2/5], Step [300/391], Loss: 0.8674
Epoch [2/5], Step [300/391], Loss: 0.8674
Muon | Epoch 2, avg loss: 1.0447, test acc: 70.49%, time: 18.92 seconds
Muon | Epoch 2, avg loss: 1.0447, test acc: 70.49%, time: 18.92 seconds
Epoch [3/5], Step [100/391], Loss: 0.8537
Epoch [3/5], Step [100/391], Loss: 0.8537
Epoch [3/5], Step [200/391], Loss: 0.9549
Epoch [3/5], Step [200/391], Loss: 0.9549
Epoch [3/5], Step [300/391], Loss: 0.7382
Epoch [3/5], Step [300/391], Loss: 0.7382
Muon | Epoch 3, avg loss: 0.8655, test acc: 75.04%, time: 18.06 seconds
Muon | Epoch 3, avg loss: 0.8655, test acc: 75.04%, time: 18.06 seconds
Epoch [4/5], Step [100/391], Loss: 0.6497
Epoch [4/5], Step [100/391], Loss: 0.6497
Epoch [4/5], Step [200/391], Loss: 0.6223

```

```
Epoch [4/5], Step [200/391], Loss: 0.6223
Epoch [4/5], Step [300/391], Loss: 0.6734
Epoch [4/5], Step [300/391], Loss: 0.6734
Muon | Epoch 4, avg loss: 0.7297, test acc: 77.86%, time: 18.85 seconds
Muon | Epoch 4, avg loss: 0.7297, test acc: 77.86%, time: 18.85 seconds
Epoch [5/5], Step [100/391], Loss: 0.7073
Epoch [5/5], Step [100/391], Loss: 0.7073
Epoch [5/5], Step [200/391], Loss: 0.6385
Epoch [5/5], Step [200/391], Loss: 0.6385
Epoch [5/5], Step [300/391], Loss: 0.5542
Epoch [5/5], Step [300/391], Loss: 0.5542
Muon | Epoch 5, avg loss: 0.6547, test acc: 80.07%, time: 18.60 seconds
Muon | Total training time: 94.45 seconds
```

--- Training with SGD ---

```
Muon | Epoch 5, avg loss: 0.6547, test acc: 80.07%, time: 18.60 seconds
Muon | Total training time: 94.45 seconds
```

--- Training with SGD ---

```
Epoch [1/5], Step [100/391], Loss: 1.9261
Epoch [1/5], Step [100/391], Loss: 1.9261
Epoch [1/5], Step [200/391], Loss: 1.8687
Epoch [1/5], Step [200/391], Loss: 1.8687
Epoch [1/5], Step [300/391], Loss: 1.5325
Epoch [1/5], Step [300/391], Loss: 1.5325
SGD | Epoch 1, avg loss: 1.8521, test acc: 46.18%, time: 16.16 seconds
SGD | Epoch 1, avg loss: 1.8521, test acc: 46.18%, time: 16.16 seconds
Epoch [2/5], Step [100/391], Loss: 1.6299
Epoch [2/5], Step [100/391], Loss: 1.6299
Epoch [2/5], Step [200/391], Loss: 1.7031
Epoch [2/5], Step [200/391], Loss: 1.7031
Epoch [2/5], Step [300/391], Loss: 1.4604
Epoch [2/5], Step [300/391], Loss: 1.4604
SGD | Epoch 2, avg loss: 1.4756, test acc: 55.07%, time: 16.62 seconds
SGD | Epoch 2, avg loss: 1.4756, test acc: 55.07%, time: 16.62 seconds
Epoch [3/5], Step [100/391], Loss: 1.2055
Epoch [3/5], Step [100/391], Loss: 1.2055
Epoch [3/5], Step [200/391], Loss: 1.3328
Epoch [3/5], Step [200/391], Loss: 1.3328
Epoch [3/5], Step [300/391], Loss: 1.1551
Epoch [3/5], Step [300/391], Loss: 1.1551
SGD | Epoch 3, avg loss: 1.2916, test acc: 60.63%, time: 16.02 seconds
SGD | Epoch 3, avg loss: 1.2916, test acc: 60.63%, time: 16.02 seconds
Epoch [4/5], Step [100/391], Loss: 1.1793
Epoch [4/5], Step [100/391], Loss: 1.1793
Epoch [4/5], Step [200/391], Loss: 1.0660
Epoch [4/5], Step [200/391], Loss: 1.0660
Epoch [4/5], Step [300/391], Loss: 1.0069
Epoch [4/5], Step [300/391], Loss: 1.0069
SGD | Epoch 4, avg loss: 1.1500, test acc: 64.74%, time: 16.37 seconds
SGD | Epoch 4, avg loss: 1.1500, test acc: 64.74%, time: 16.37 seconds
Epoch [5/5], Step [100/391], Loss: 1.1009
Epoch [5/5], Step [100/391], Loss: 1.1009
```

```
Epoch [5/5], Step [200/391], Loss: 1.0298
Epoch [5/5], Step [200/391], Loss: 1.0298
Epoch [5/5], Step [300/391], Loss: 1.1095
Epoch [5/5], Step [300/391], Loss: 1.1095
SGD | Epoch 5, avg loss: 1.0681, test acc: 65.82%, time: 16.27 seconds
SGD | Total training time: 81.44 seconds
```

--- Training with AdamW ---

```
SGD | Epoch 5, avg loss: 1.0681, test acc: 65.82%, time: 16.27 seconds
SGD | Total training time: 81.44 seconds
```

--- Training with AdamW ---

```
Epoch [1/5], Step [100/391], Loss: 1.7724
Epoch [1/5], Step [100/391], Loss: 1.7724
Epoch [1/5], Step [200/391], Loss: 1.4650
Epoch [1/5], Step [200/391], Loss: 1.4650
Epoch [1/5], Step [300/391], Loss: 1.6820
Epoch [1/5], Step [300/391], Loss: 1.6820
AdamW | Epoch 1, avg loss: 1.6505, test acc: 54.74%, time: 16.37 seconds
AdamW | Epoch 1, avg loss: 1.6505, test acc: 54.74%, time: 16.37 seconds
Epoch [2/5], Step [100/391], Loss: 1.2562
Epoch [2/5], Step [100/391], Loss: 1.2562
Epoch [2/5], Step [200/391], Loss: 1.4044
Epoch [2/5], Step [200/391], Loss: 1.4044
Epoch [2/5], Step [300/391], Loss: 1.1770
Epoch [2/5], Step [300/391], Loss: 1.1770
AdamW | Epoch 2, avg loss: 1.2823, test acc: 62.07%, time: 16.38 seconds
AdamW | Epoch 2, avg loss: 1.2823, test acc: 62.07%, time: 16.38 seconds
Epoch [3/5], Step [100/391], Loss: 1.2210
Epoch [3/5], Step [100/391], Loss: 1.2210
Epoch [3/5], Step [200/391], Loss: 1.0531
Epoch [3/5], Step [200/391], Loss: 1.0531
Epoch [3/5], Step [300/391], Loss: 0.9832
Epoch [3/5], Step [300/391], Loss: 0.9832
AdamW | Epoch 3, avg loss: 1.0970, test acc: 67.36%, time: 16.36 seconds
AdamW | Epoch 3, avg loss: 1.0970, test acc: 67.36%, time: 16.36 seconds
Epoch [4/5], Step [100/391], Loss: 0.9645
Epoch [4/5], Step [100/391], Loss: 0.9645
Epoch [4/5], Step [200/391], Loss: 0.9276
Epoch [4/5], Step [200/391], Loss: 0.9276
Epoch [4/5], Step [300/391], Loss: 0.9332
Epoch [4/5], Step [300/391], Loss: 0.9332
AdamW | Epoch 4, avg loss: 0.9764, test acc: 69.80%, time: 16.15 seconds
AdamW | Epoch 4, avg loss: 0.9764, test acc: 69.80%, time: 16.15 seconds
Epoch [5/5], Step [100/391], Loss: 0.8354
Epoch [5/5], Step [100/391], Loss: 0.8354
Epoch [5/5], Step [200/391], Loss: 0.7680
Epoch [5/5], Step [200/391], Loss: 0.7680
Epoch [5/5], Step [300/391], Loss: 0.9531
Epoch [5/5], Step [300/391], Loss: 0.9531
AdamW | Epoch 5, avg loss: 0.9061, test acc: 71.14%, time: 16.80 seconds
AdamW | Total training time: 82.06 seconds
```

--- Training with MuonSVD ---

AdamW | Epoch 5, avg loss: 0.9061, test acc: 71.14%, time: 16.80 seconds
AdamW | Total training time: 82.06 seconds

--- Training with MuonSVD ---

```
/tmp/ipython-input-3592678830.py:15: UserWarning: torch.linalg.svd:
During SVD computation with the selected cusolver driver, batches 0
failed to converge. A more accurate method will be used to compute the
SVD as a fallback. Check doc at
https://pytorch.org/docs/stable/generated/torch.linalg.svd.html
(Triggered internally at
/pytorch/aten/src/ATen/native/cuda/linalg/BatchLinearAlgebraLib.cpp:70
2.)
```

```
U, _, Vh = torch.linalg.svd(update, full_matrices=False)
```

```
Epoch [1/5], Step [100/391], Loss: 1.7036
Epoch [1/5], Step [200/391], Loss: 1.4701
Epoch [1/5], Step [200/391], Loss: 1.4701
Epoch [1/5], Step [300/391], Loss: 0.9398
Epoch [1/5], Step [300/391], Loss: 0.9398
MuonSVD | Epoch 1, avg loss: 1.4415, test acc: 66.81%, time: 25.62
seconds
MuonSVD | Epoch 1, avg loss: 1.4415, test acc: 66.81%, time: 25.62
seconds
Epoch [2/5], Step [100/391], Loss: 1.0737
Epoch [2/5], Step [100/391], Loss: 1.0737
Epoch [2/5], Step [200/391], Loss: 0.8619
Epoch [2/5], Step [200/391], Loss: 0.8619
Epoch [2/5], Step [300/391], Loss: 0.9035
Epoch [2/5], Step [300/391], Loss: 0.9035
MuonSVD | Epoch 2, avg loss: 1.0040, test acc: 73.51%, time: 23.99
seconds
MuonSVD | Epoch 2, avg loss: 1.0040, test acc: 73.51%, time: 23.99
seconds
Epoch [3/5], Step [100/391], Loss: 0.8254
Epoch [3/5], Step [100/391], Loss: 0.8254
Epoch [3/5], Step [200/391], Loss: 0.8481
Epoch [3/5], Step [200/391], Loss: 0.8481
Epoch [3/5], Step [300/391], Loss: 0.7257
Epoch [3/5], Step [300/391], Loss: 0.7257
MuonSVD | Epoch 3, avg loss: 0.8505, test acc: 76.47%, time: 23.87
seconds
MuonSVD | Epoch 3, avg loss: 0.8505, test acc: 76.47%, time: 23.87
seconds
Epoch [4/5], Step [100/391], Loss: 0.6814
Epoch [4/5], Step [100/391], Loss: 0.6814
Epoch [4/5], Step [200/391], Loss: 0.7314
Epoch [4/5], Step [200/391], Loss: 0.7314
Epoch [4/5], Step [300/391], Loss: 0.6497
Epoch [4/5], Step [300/391], Loss: 0.6497
MuonSVD | Epoch 4, avg loss: 0.7230, test acc: 79.03%, time: 23.87
```

seconds

MuonSVD | Epoch 4, avg loss: 0.7230, test acc: 79.03%, time: 23.87

seconds

Epoch [5/5], Step [100/391], Loss: 0.5782

Epoch [5/5], Step [100/391], Loss: 0.5782

Epoch [5/5], Step [200/391], Loss: 0.6392

Epoch [5/5], Step [200/391], Loss: 0.6392

Epoch [5/5], Step [300/391], Loss: 0.5670

Epoch [5/5], Step [300/391], Loss: 0.5670

MuonSVD | Epoch 5, avg loss: 0.6410, test acc: 80.63%, time: 23.76

seconds

MuonSVD | Total training time: 121.10 seconds

--- Training with AdamWMuP ---

MuonSVD | Epoch 5, avg loss: 0.6410, test acc: 80.63%, time: 23.76

seconds

MuonSVD | Total training time: 121.10 seconds

--- Training with AdamWMuP ---

Epoch [1/5], Step [100/391], Loss: 1.7013

Epoch [1/5], Step [100/391], Loss: 1.7013

Epoch [1/5], Step [200/391], Loss: 1.5538

Epoch [1/5], Step [200/391], Loss: 1.5538

Epoch [1/5], Step [300/391], Loss: 1.3699

Epoch [1/5], Step [300/391], Loss: 1.3699

AdamWMuP | Epoch 1, avg loss: 1.6242, test acc: 54.07%, time: 17.52

seconds

AdamWMuP | Epoch 1, avg loss: 1.6242, test acc: 54.07%, time: 17.52

seconds

Epoch [2/5], Step [100/391], Loss: 1.2454

Epoch [2/5], Step [100/391], Loss: 1.2454

Epoch [2/5], Step [200/391], Loss: 1.2509

Epoch [2/5], Step [200/391], Loss: 1.2509

Epoch [2/5], Step [300/391], Loss: 1.3320

Epoch [2/5], Step [300/391], Loss: 1.3320

AdamWMuP | Epoch 2, avg loss: 1.2549, test acc: 62.31%, time: 16.33

seconds

AdamWMuP | Epoch 2, avg loss: 1.2549, test acc: 62.31%, time: 16.33

seconds

Epoch [3/5], Step [100/391], Loss: 1.0594

Epoch [3/5], Step [100/391], Loss: 1.0594

Epoch [3/5], Step [200/391], Loss: 1.0099

Epoch [3/5], Step [200/391], Loss: 1.0099

Epoch [3/5], Step [300/391], Loss: 1.0086

Epoch [3/5], Step [300/391], Loss: 1.0086

AdamWMuP | Epoch 3, avg loss: 1.0591, test acc: 68.42%, time: 17.36

seconds

AdamWMuP | Epoch 3, avg loss: 1.0591, test acc: 68.42%, time: 17.36

seconds

Epoch [4/5], Step [100/391], Loss: 0.8411

Epoch [4/5], Step [100/391], Loss: 0.8411

Epoch [4/5], Step [200/391], Loss: 0.9039

Epoch [4/5], Step [200/391], Loss: 0.9039

Epoch [4/5], Step [300/391], Loss: 0.9845

Epoch [4/5], Step [300/391], Loss: 0.9845

AdamWMuP | Epoch 4, avg loss: 0.9462, test acc: 70.14%, time: 16.47 seconds

AdamWMuP | Epoch 4, avg loss: 0.9462, test acc: 70.14%, time: 16.47 seconds

Epoch [5/5], Step [100/391], Loss: 0.9703

Epoch [5/5], Step [100/391], Loss: 0.9703

Epoch [5/5], Step [200/391], Loss: 0.8429

Epoch [5/5], Step [200/391], Loss: 0.8429

Epoch [5/5], Step [300/391], Loss: 0.8273

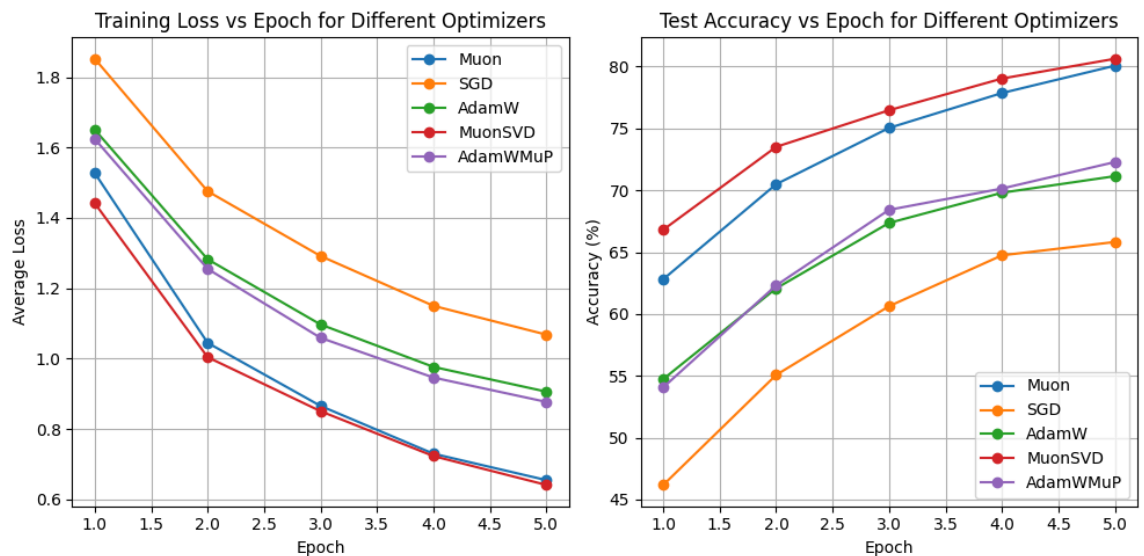
Epoch [5/5], Step [300/391], Loss: 0.8273

AdamWMuP | Epoch 5, avg loss: 0.8769, test acc: 72.28%, time: 17.05 seconds

AdamWMuP | Total training time: 84.73 seconds

AdamWMuP | Epoch 5, avg loss: 0.8769, test acc: 72.28%, time: 17.05 seconds

AdamWMuP | Total training time: 84.73 seconds



Question 3

Which optimizer performed best between Muon, SGD, and AdamW? Also copy the resulting plots into the submission as well.

Question 4

Compare the loss curves between Muon and MuonSVD. Are the results expected? Explain why.

Question 5

The Muon optimizer contains two key differences: (1) orthogonalization of the momentum, and (2) muP scaling of the momentum. **Between orthogonalization and muP scaling, which seemed to matter more?** Reference the loss curves to justify your answer.

Answer questions 3,4,5 based on the outcomes of the previous cells

Let me get the latest notebook summary and check the outputs from the training cells.

Retrieving Notebook summary.

Read output of `[(vscode-notebook-cell:/Users/azmac98/Downloads/hw6/q_coding_muon.ipynb#X22sZmlsZQ%3D%3D)]`

Now I have all the data needed to answer questions 3, 4, and 5.

Answer to Question 3

Which optimizer performed best between Muon, SGD, and AdamW?

Muon performed the best among the three main optimizers.

Optimizer	Final Test Accuracy (Epoch 5)	Final Avg Loss
Muon	80.07%	0.6547
AdamW	71.14%	0.9061
SGD	65.82%	1.0681

Muon achieved significantly higher test accuracy (80.07%) compared to AdamW (71.14%) and SGD (65.82%). The loss curve also shows Muon converging faster and to a lower final loss value. This demonstrates the benefits of orthogonalizing the momentum update, which allows the optimizer to utilize all directions in parameter space more effectively.

Answer to Question 4

Compare the loss curves between Muon and MuonSVD. Are the results expected? Explain why.

Yes, the results are expected. The loss curves for Muon and MuonSVD are nearly identical:

Optimizer	Final Test Accuracy	Final Avg Loss
Muon	80.07%	0.6547
MuonSVD	80.63%	0.6410

This is expected because **both methods achieve the same mathematical goal: orthogonalization of the momentum matrix.**

- **MuonSVD** computes UV^T directly from the SVD decomposition $M = U\Sigma V^T$, which gives an exact orthogonal matrix.
- **Muon (Newton-Schulz)** iteratively approximates the same orthogonal matrix by driving all singular values toward 1.

Since both approaches produce (approximately) the same orthogonalized update, the training dynamics and final results are very similar. The main difference is computational: Newton-Schulz is faster (~94s vs ~121s) because it avoids the expensive SVD computation, making Muon more practical for large-scale training.

Answer to Question 5

Between orthogonalization and muP scaling, which seemed to matter more?

Orthogonalization mattered more than muP scaling.

Evidence from the results:

Optimizer	Key Difference	Final Test Accuracy
Muon	Orthogonalization + muP scaling	80.07%
MuonSVD	Orthogonalization + muP scaling	80.63%
AdamWMuP	muP scaling only (no orthogonalization)	72.28%

| AdamW | Neither | 71.14% |

Comparing the curves:

- **AdamW → AdamWMuP** (adding muP scaling): Only improved accuracy from 71.14% to 72.28% (+1.14%)
- **AdamW → Muon** (adding orthogonalization + muP): Improved accuracy from 71.14% to 80.07% (+8.93%)

The muP scaling alone (AdamWMuP) provides only a marginal improvement (~1%) over standard AdamW. In contrast, the orthogonalization step provides the dominant benefit, accounting for most of the ~9% improvement. This shows that **making the gradient updates act uniformly across all directions in parameter space (via orthogonalization) is the key innovation in Muon**, while the muP scaling is a secondary refinement for consistent update magnitudes.

Question 6 (Optional)

Our implementation of Newton-Schulz is suboptimal in the polynomial used for convergence. The community has developed quintic polynomials that converge faster while still being efficient. **Implement an improved Newton-Schulz and compare. Comment on the speed advantage of the improved Muon relative to the MuonSVD.**

(Hint: You can modify the number of iterations by setting the `ns_iters` parameter in the Muon optimizer)

Part 4 (Optional): Hyperparameter sweeps

To ensure that we are making fair comparisons, we will sweep over both learning rate and weight decay for both Muon and AdamW (the likely strongest competing optimizer). We choose these two parameters because empirically, they have the greatest effect on training.

Running the sweep should take less than 1 hour total on a GPU runtime.

```
In [ ]: from torch.utils.data import random_split

device = "cuda" if torch.cuda.is_available() else "cpu"
criterion = nn.CrossEntropyLoss()

# Function to evaluate model
def evaluate(model, dataloader):
```

```

model.eval()
correct = 0
total = 0
with torch.no_grad():
    for images, labels in dataloader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
return 100 * correct / total

def train_and_evaluate(optimizer_fn, lr, weight_decay, num_epochs=5):
    """Trains and evaluates the CIFAR10CNN with a given optimizer and
    hyperparameters."""
    model = CIFAR10CNN().to(device)
    optimizer = optimizer_fn(model.parameters(), lr=lr,
weight_decay=weight_decay)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=num_epochs)

    best_accuracy = 0

    # Split training data into training and validation sets
    train_size = int(0.8 * len(train_ds))
    val_size = len(train_ds) - train_size
    train_dataset, val_dataset = random_split(train_ds, [train_size,
val_size])

    train_loader_split = DataLoader(train_dataset, batch_size=batch_size,
shuffle=True, num_workers=2)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False,
num_workers=2)

    for epoch in range(1, num_epochs + 1):
        model.train()
        for i, (x, y) in enumerate(train_loader_split):
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad()
            out = model(x)
            loss = criterion(out, y)
            loss.backward()
            optimizer.step()

        # Evaluate on the validation set
        val_acc = evaluate(model, val_loader)
        best_accuracy = max(best_accuracy, val_acc)

        scheduler.step()

    return best_accuracy

```

```

# Define the optimizers and their hyperparameter search spaces
optimizers_to_sweep = {
    "Muon": {
        "optimizer_fn": lambda params, lr, weight_decay: Muon(params, lr=lr,
weight_decay=weight_decay),
        "lr_values": [1e-2, 5e-3, 1e-3, 5e-4],
        "weight_decay_values": [0, 1e-4, 1e-3]
    },
    "AdamW": {
        "optimizer_fn": lambda params, lr, weight_decay:
torch.optim.AdamW(params, lr=lr, weight_decay=weight_decay),
        "lr_values": [1e-2, 5e-3, 1e-3, 5e-4],
        "weight_decay_values": [0, 1e-4, 1e-3]
    }
}

results = {}

# Perform the hyperparameter sweep for each optimizer
for opt_name, opt_info in optimizers_to_sweep.items():
    print(f"\n--- Hyperparameter sweep for {opt_name} ---")
    optimizer_fn = opt_info["optimizer_fn"]
    lr_values = opt_info["lr_values"]
    weight_decay_values = opt_info["weight_decay_values"]

    results[opt_name] = {}

    for lr in lr_values:
        for weight_decay in weight_decay_values:
            print(f"Training with lr={lr}, weight_decay={weight_decay}")
            accuracy = train_and_evaluate(optimizer_fn, lr, weight_decay)
            results[opt_name][(lr, weight_decay)] = accuracy
            print(f"Validation Accuracy: {accuracy:.2f}%")

# Print the results and best hyperparameters for each optimizer
print("\n--- Hyperparameter Sweep Results ---")
for opt_name, opt_results in results.items():
    print(f"\n{opt_name}:")
    for (lr, weight_decay), accuracy in opt_results.items():
        print(f"  (lr={lr}, weight_decay={weight_decay}): {accuracy:.2f}%")

    best_params = max(opt_results, key=opt_results.get)
    print(f"Best hyperparameters for {opt_name}: (lr={best_params[0]},
weight_decay={best_params[1]}) with validation accuracy
{opt_results[best_params]:.2f}%")

```

Question 7 (Optional)

What were the best choices for hyperparameters for Muon? What about for AdamW?**In []:**

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).